DOCUMENTATION · SECURITY

Where data goes

The latest version is at anantharajuc.github.io/QueryAPIGate-website/Guide.dc.html?d=DATA_PROTECTION. This copy was made on 4 November 2026.

What QueryAPIGate keeps, where, for how long, who can read it - and what it never keeps. For a record of processing (GDPR Article 30), a data-classification review, or a security questionnaire's data section (ADR 0016). Every table of the store, every setting that bounds what is kept, and every place data leaves the process is on this page: a test fails if one isn't. Walkthrough, answering a questionnaire from a running server: Answer a security questionnaire about data and cryptography.

What passes through, and isn't kept

QueryAPIGate sits between callers and databases. **The rows a query returns pass through and aren't kept**: they're streamed or paged to the caller, after column masking, and written nowhere - with two exceptions you turn on: the response cache and exports.

Parameter values aren't kept either. A saved query's parameters are bound, not written into the SQL; run history records which query ran and with which parameter *names*, never their values; traces never hold a value (ADR 0010); the access log leaves the query string out (`QUERYAPIGATE_ACCESS_LOG_QUERY` , below).

Personal data it handles

CATEGORY	WHERE IT COMES FROM	WHY IT'S KEPT
Callers' names	API key names; `jwt:<subject>` for a signed-in user - often an email address	To say who ran what, apply their limits and quotas
Administrators' names and email addresses	Created by an owner, or from SSO	To say who changed what
IP addresses	Every request	Rate limits, IP allow-lists, the access log, traces
Whatever the connected databases hold	Your databases	Passes through - kept only where listed below
An agent's write arguments	An AI agent asking to write (ADR 0008)	So a person can approve exactly what runs

The store

`queryapigate.db` , or the PostgreSQL store. Read through the Management API by administrators, each to what their role's capabilities allow (Administrators and admin roles).

TABLE	HOLDS	PERSONAL DATA	KEPT FOR	READ BY
`connections`	Each connection's settings; its password encrypted (`QUERYAPIGATE_SECRET_KEY`) or a reference	-	Until deleted	`connections.read` (passwords masked)
`saved_queries`, `saved_query_versions`	Saved queries and every version: SQL, parameters, descriptions, the author's name	Author	Until deleted	`queries.read`
`descriptions`	Descriptions of tables and columns, and who wrote each	Administrator's name	Until changed	`connections.read`; agents
`execution_history`	Each run: when, which query and version, the caller, the transport, rows, time, status, QueryAPIGate's error message; an ad-hoc run's parameter *names* and its SQL as `QUERYAPIGATE_HISTORY_ADHOC_SQL` says (`text`, `hash`, `none`)	Caller	The newest `QUERYAPIGATE_HISTORY_LIMIT` runs per version and `QUERYAPIGATE_HISTORY_ADHOC_LIMIT` ad-hoc runs - or `QUERYAPIGATE_HISTORY_RETENTION_DAYS` days; `QUERYAPIGATE_HISTORY_SAMPLE_RATE` keeps only a share	`observe`
`events`	The event log (ADR 0020): each run as `/events` sends it - the same fields as its `execution_history` entry, every run whatever the sample rate	Caller	`QUERYAPIGATE_EVENTS_RETENTION_HOURS` (24 by default)	Each key its own events, through `/events`; the admin key every one
`usage_counters`	Calls and rows per caller per day and month (quotas, ADR 0011)	Caller	400 days	`observe`
`audit_log`, `audit_chain`, `audit_delivery`	Every administrative change: who, how they signed in, what changed (passwords only as "changed"); its hash chain; how far each SIEM output has got	Administrator's name	The newest `QUERYAPIGATE_AUDIT_LOG_LIMIT` entries - more while a SIEM output is behind	`observe`
`api_keys`, `roles`	Keys' names, grants and the SHA-256 of their secrets; when each was last used	Key names	Until deleted	`access.read` (never a secret)
`administrators`, `admin_tokens`	Administrators' names, roles, email addresses, SSO subjects, when last seen; their tokens' hashes	Name, email	Until deleted	`admins.read`
`workspaces`, `workspace_members`	Each workspace's name and description; which administrators belong to which, with what role, added by whom (ADR 0023)	Administrators' names	Until deleted - a member until removed or their administrator is deleted	`admins.read`

TABLE	HOLDS	PERSONAL DATA	KEPT FOR	READ BY
`admin_sessions`	Console sessions through SSO, by the hash of their id	Administrator's name	`QUERYAPIGATE_SSO_SESSION_HOURS`	-
`sso_logins`	A sign-in under way: its state and PKCE verifier	-	10 minutes	-
`write_requests`	An agent's write: its arguments, who asked, who approved or rejected it, the result	Caller, approver; the arguments may be personal	30 days	`writes.approve`
`destinations`, `exports`	Where exports go, with credentials encrypted or as references; each export's watermark - the last value of its incremental column it delivered	The watermark may be a value	Until deleted	`exports.read` (secrets masked)
`instances`	Each running process: host name, process id, version	-	1 day after it stops	`observe`
`config_managed`, `config_applied`	What a configuration file manages, and each file applied, by its hash (ADR 0014)	-	Until deleted	`observe`
`server_secrets`	Keys the server makes for itself - masking's `hash` key, unless `QUERYAPIGATE_MASK_KEY` is set	-	For the store's life	-
`run_store`, `run_writer_leases`	Where runs are kept, if not here (ADR 0029): the ClickHouse server and database, never its credentials; which process holds which block of event ids, by host name and process id	-	`run_store` until changed; a lease 60 seconds after its holder stops	-
`schema_version`	The store's schema version	-	-	-

The run store (ClickHouse, optional)

With `QUERYAPIGATE_RUN_STORE_URL` set (ADR 0029), `execution_history` and `events` above stay empty: runs and events are kept in ClickHouse instead, with the same fields.

TABLE	HOLDS	PERSONAL DATA	KEPT FOR	READ BY
`events`	The event log: every run, whatever the sample rate, and its history entry	Caller	`QUERYAPIGATE_EVENTS_RETENTION_HOURS` (24 by default)	As the event log above
`history`	The runs history keeps (`QUERYAPIGATE_HISTORY_SAMPLE_RATE`)	Caller	`QUERYAPIGATE_HISTORY_RETENTION_DAYS` (30 by default here)	`observe`
`history_counts`	Runs history holds and the last of them, per day, query, version and caller	Caller	As `history`	`observe`
`run_minutes`	Runs, rows and query times per minute, by caller, query, version, connection, transport, status, error code and ad-hoc statement	Caller; a statement may hold values	As `events`	`observe`
`run_hours`	The same per hour, with each ad-hoc statement as its SHA-256, not its text	Caller	400 days	- (ClickHouse only)

`forget` erases a caller from all five: hidden at once, and gone from disk once ClickHouse has rewritten the parts that held them. It deletes a second time a few seconds later, for runs that were still on their way.

The response cache and Redis

WHERE	HOLDS	KEPT FOR
The response cache (in the process, or Redis with `QUERYAPIGATE_REDIS_URL`)	Responses of saved queries that set `cache_ttl` - **rows**, after masking - by query, version, parameter values, format and page; never by caller	The query's `cache_ttl`
Redis's rate-limit counters	Calls per key and per client address in the current window	The window
Redis's quota counters	Calls and rows per caller per period	40 days
Redis's limits	Queries running now, per caller (ADR 0011)	While they run

Where data is sent

TO	WHAT	BOUNDED BY
The log (stdout)	One line per request: method, path, status, time, the caller's name, the request id - **not the query string**; each statement's SQL, or with `QUERYAPIGATE_HISTORY_ADHOC_SQL` `hash` or `none`, its SHA-256; warnings and errors - a database's own error message among them, which can quote a value. `QUERYAPIGATE_JSON_LOGS` for JSON	Your log platform
The access log (the image's gunicorn, `queryapigate serve`)	Client address, time, method, **path without the query string**, status, size, time taken, request id, user agent - never the referrer. `QUERYAPIGATE_ACCESS_LOG_QUERY=1` keeps the query string	Your log platform
Traces (`OTEL_EXPORTER_OTLP_ENDPOINT`)	Spans: the route, the saved query's name, rows, the caller's name and address - never a parameter value, row, statement or credential (Tracing)	Your collector
`/metrics`	Counts and timings by connection, query and API key name; every signed-in user under one `jwt` label, every administrator under one `admin`	Your Prometheus - and whoever can reach `/metrics`, unless `QUERYAPIGATE_METRICS_TOKEN` is set (the Helm chart sets it)
The audit log's copies (`QUERYAPIGATE_AUDIT_LOG_EXPORT_FILE`, `QUERYAPIGATE_AUDIT_SYSLOG_URL`, `QUERYAPIGATE_AUDIT_OTLP`)	Every audit entry, with its place in the chain	Your file and SIEM
Live events (`/events`)	Each run as it happens: the same fields as run history	Not kept
AI agents (MCP)	Rows, after masking, up to `QUERYAPIGATE_MCP_MAX_ROWS` a call; descriptions	What the agent keeps
Exports (destinations)	A saved query's rows, after the export's masking, to the bucket or folder it names	Your storage
Secret providers (Vault, AWS)	Requests for the secrets connections name (ADR 0015)	-

What a deployer controls

- **Run history:** how much is kept (`QUERYAPIGATE_HISTORY_LIMIT`, `QUERYAPIGATE_HISTORY_ADHOC_LIMIT`, `QUERYAPIGATE_HISTORY_RETENTION_DAYS`), whether every run is (`QUERYAPIGATE_HISTORY_SAMPLE_RATE`), and what of an ad-hoc statement (`QUERYAPIGATE_HISTORY_ADHOC_SQL`).
- **The audit log:** how many entries the store keeps (`QUERYAPIGATE_AUDIT_LOG_LIMIT`); its copies elsewhere are yours to keep for as long as your obligations say.
- **What leaves:** column masking, by key, role and export; the response cache, per query; the access log's query strings; tracing, when an endpoint is set.
- **A person's data, on request:** below.

A person asks for their data to be erased

A person who signs in to call QueryAPIGate is named `jwt:<subject>` - their email address, often - in run history and usage counts. To erase them:

```shell
queryapigate history forget jwt:alice@example.com --dry-run   # how many runs, events and usage periods
queryapigate history forget jwt:alice@example.com
```

or `DELETE /api/v1/history?caller=jwt:alice@example.com` (owners only, `history.forget`). Their runs, events and usage counts - in the store and in Redis - are deleted, and the erasure is recorded in the audit log as `forget_caller` with the caller's SHA-256 and the counts, not their name. That record is pseudonymous: someone who already knows the name can confirm that it was erased - which is how you show a request was met - but it names no one to anyone who doesn't. Their quota's count for the period starts again.

With several instances, runs another instance has queued but not yet written - for at most `QUERYAPIGATE_HISTORY_FLUSH_INTERVAL`, a second by default - can land just after. Check with `--dry-run` a few seconds later, or run it again: it finds what arrived since.

What isn't erased, and what to say:

- **The audit log** names administrators acting in their role - keep it under legitimate interest, for as long as your accountability obligations require. It is hash-chained (ADR 0012): an entry can't be removed without breaking the chain. A person who isn't an administrator isn't in it.
- **Write requests** - an agent's, naming the caller - are kept 30 days, then deleted.
- **Logs, traces and SIEM copies** are in your platforms, under their retention.
- **The connected databases** are yours: QueryAPIGate doesn't hold their data.

Cryptography

Every algorithm QueryAPIGate uses is FIPS-approved (FIPS 140-3, SP 800-131A); a test fails if the code gains one that isn't - MD5, SHA-1, BLAKE2, or `random` for anything that must be unpredictable.

PROTECTS	ALGORITHM	THROUGH
Connection and destination passwords at rest (`QUERYAPIGATE_SECRET_KEY`)	Fernet: AES-128-CBC and HMAC-SHA256	`cryptography`
API keys, admin tokens, Console sessions, sign-in state - stored as hashes	SHA-256	`hashlib`
The audit log's chain (`QUERYAPIGATE_AUDIT_KEY`)	HMAC-SHA256 (SHA-256 without a key)	`hmac`, `hashlib`
Column masking's `hash`	HMAC-SHA256	`hmac`
PKCE for Console sign-in	SHA-256 (`S256`)	`hashlib`
Secrets it makes - keys, tokens, verifiers, ids	The operating system's random source	`secrets`
Signed-in users' tokens	RS256 with a JWKS, HS256 with a shared secret - or `QUERYAPIGATE_JWT_ALGORITHMS` (keep to RS*, PS*, ES*, HS*)	PyJWT, `cryptography`
TLS: to Vault, the SIEM, the OTLP collector, Redis	As the platform's OpenSSL negotiates	Python's `ssl`
RDS IAM tokens	AWS Signature Version 4: HMAC-SHA256	botocore
Cache keys, ETags, configuration files' hashes - not secrets	SHA-256	`hashlib`

`random` is used only where nothing depends on it being unpredictable: which runs history samples, and the example database, from a fixed seed.

On a FIPS-validated module

Validation is a property of the cryptographic module, not of the application: QueryAPIGate is FIPS-ready when it runs on a validated OpenSSL in FIPS mode, and that is a choice of platform.

- **Use a distribution whose OpenSSL is validated, in FIPS mode** - Red Hat Enterprise Linux 9 (booted with `fips=1`, or `fips-mode-setup --enable`), or Ubuntu Pro FIPS. A container uses its host's mode.
- **Python and `cryptography` from the distribution, not PyPI** - PyPI's `cryptography` wheel carries its own OpenSSL, which is not a validated module. On RHEL 9:

  ```shell
  dnf install python3.12 python3.12-pip python3.12-cryptography
  python3.12 -m venv --system-site-packages /opt/queryapigate
  /opt/queryapigate/bin/pip install "queryapigate[server,postgres]"   # keeps the distribution's cryptography
  ```

- **Check it:** both must say so.

```shell
/opt/queryapigate/bin/python -c "import hashlib; hashlib.md5(b'x')"           # ValueError: unsupported
/opt/queryapigate/bin/python -c "from cryptography.hazmat.backends.openssl import backend; print(backend._fips_enabled)"
```

- **Database drivers bring their own TLS.** `psycopg2-binary` carries its own libpq and OpenSSL: on a FIPS platform, build `psycopg2` from source against the system's libpq (`dnf install libpq-devel gcc python3.12-devel`, then `pip install psycopg2 --no-binary psycopg2`). `mysql-connector-python`'s C extension carries its own OpenSSL too: MySQL connections are outside the validated boundary unless your connector build uses the system's. Also outside QueryAPIGate's control: DuckDB's own TLS for remote files, and a JDBC or H2 connection's Java runtime - use a FIPS-configured JRE, or don't use them there.
- **Leave the Rust data plane off** (experimental, ADR 0030). Its cryptography - the Fernet passwords it decrypts, the SHA-256 and HMAC it hashes keys and masks with, its TLS - is ring and RustCrypto, not a validated module. It runs only when started (`dataplane.enabled` in the chart), so a deployment without it is unchanged.
- **What your platform's FIPS policy refuses is refused** - a JWT algorithm in `QUERYAPIGATE_JWT_ALGORITHMS` its module doesn't offer, a TLS cipher it doesn't allow - with an error, never a weaker fallback.

Tested in CI: a job runs the whole suite in Red Hat UBI 9 with OpenSSL's FIPS provider forced (`OPENSSL_FORCE_FIPS_MODE=1`), Python 3.12 and `cryptography` from the distribution - and first checks that FIPS is really in force: MD5 refused, `cryptography` on the system's OpenSSL, in FIPS mode. RHEL 9's SQLite is 3.34, older than the 3.35 that `RETURNING` and `DROP COLUMN` need: an agent's write with `RETURNING` to a SQLite database fails there, and the two tests that need 3.35 skip.